

Análise de Fluxo de Dados



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismailly
ismailybf@ufc.br

Semestre 20025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)

Introdução

- Otimizações:

Introdução

- Otimizações:
 - São transformações para ganho de eficiência.

Introdução

- Otimizações:
 - São transformações para ganho de eficiência.
 - Não podem alterar a saída do programa.

Introdução

- Otimizações:
 - São transformações para ganho de eficiência.
 - Não podem alterar a saída do programa.
- Exemplos:

Introdução

- Otimizações:
 - São transformações para ganho de eficiência.
 - Não podem alterar a saída do programa.
- Exemplos:
 - *Dead Code Elimination* - apaga uma computação cujo resultado nunca será usado.

Introdução

- Otimizações:
 - São transformações para ganho de eficiência.
 - Não podem alterar a saída do programa.
- Exemplos:
 - *Dead Code Elimination* - apaga uma computação cujo resultado nunca será usado.
 - *Register Allocation* - reaproveitamento de registradores.

Introdução

- Otimizações:
 - São transformações para ganho de eficiência.
 - Não podem alterar a saída do programa.
- Exemplos:
 - *Dead Code Elimination* - apaga uma computação cujo resultado nunca será usado.
 - *Register Allocation* - reaproveitamento de registradores.
 - *Common-subexpression elimination* - elimina expressões redundantes, que computam o mesmo valor.

Introdução

- Otimizações:
 - São transformações para ganho de eficiência.
 - Não podem alterar a saída do programa.
- Exemplos:
 - *Dead Code Elimination* - apaga uma computação cujo resultado nunca será usado.
 - *Register Allocation* - reaproveitamento de registradores.
 - *Common-subexpression elimination* - elimina expressões redundantes, que computam o mesmo valor.
 - *Constant Folding* - se os operandos são constantes, calcule a expressão em tempo de compilação.

Introdução

- Essas transformações são feitas com base em informações coletadas do programa.

Introdução

- Essas transformações são feitas com base em informações coletadas do programa.
- Esse é o trabalho da análise de fluxo de dados (DFA).

Introdução

- Essas transformações são feitas com base em informações coletadas do programa.
- Esse é o trabalho da análise de fluxo de dados (DFA).
- *Intraprocedural global optimization*

Introdução

- Essas transformações são feitas com base em informações coletadas do programa.
- Esse é o trabalho da análise de fluxo de dados (DFA).
- *Intraprocedural global optimization*
 - Interna a um procedimento ou função.

Introdução

- Essas transformações são feitas com base em informações coletadas do programa.
- Esse é o trabalho da análise de fluxo de dados (DFA).
- *Intraprocedural global optimization*
 - Interna a um procedimento ou função.
 - Engloba todos os blocos básicos.

Análise do fluxo de dados

- Ideia básica:

Análise do fluxo de dados

- Ideia básica:
 - Coletar informações sobre o programa (*data-flow information*) para cada ponto do programa.

Análise do fluxo de dados

- Ideia básica:
 - Coletar informações sobre o programa (*data-flow information*) para cada ponto do programa.
- Informações interessantes:

Análise do fluxo de dados

- Ideia básica:
 - Coletar informações sobre o programa (*data-flow information*) para cada ponto do programa.
- Informações interessantes:
 - Definições alcançantes (*reaching definitions*).

Análise do fluxo de dados

- Ideia básica:
 - Coletar informações sobre o programa (*data-flow information*) para cada ponto do programa.
- Informações interessantes:
 - Definições alcançantes (*reaching definitions*).
 - Variáveis vivas (*Live variables*).

Análise do fluxo de dados

- Ideia básica:
 - Coletar informações sobre o programa (*data-flow information*) para cada ponto do programa.
- Informações interessantes:
 - Definições alcançantes (*reaching definitions*).
 - Variáveis vivas (*Live variables*).
 - Expressões disponíveis (*Available expressions*).

Pontos e caminhos

- Para cada instrução do programa existe um ponto p_i antes de um ponto p_{i+1} depois da instrução.

Pontos e caminhos

- Para cada instrução do programa existe um ponto p_i antes de um ponto p_{i+1} depois da instrução.
- Um caminho de p_1 até p_n é uma sequência de pontos $p_1, p_2, \dots, p_n$ tal que para cada i entre 1 e $n - 1$:

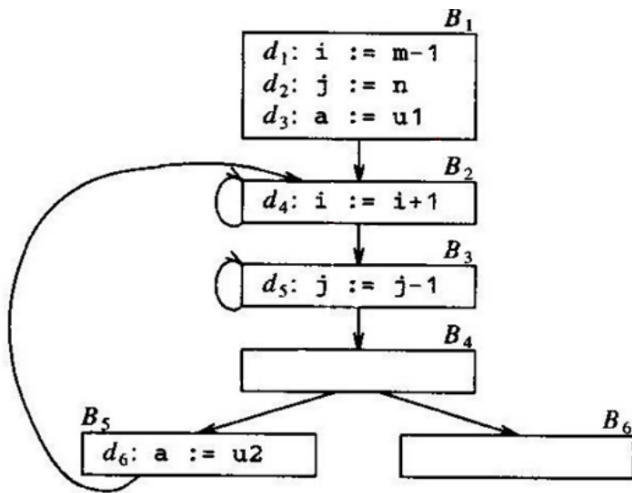
Pontos e caminhos

- Para cada instrução do programa existe um ponto p_i antes de um ponto p_{i+1} depois da instrução.
- Um caminho de p_1 até p_n é uma sequência de pontos $p_1, p_2, \dots, p_n$ tal que para cada i entre 1 e $n - 1$:
 - p_i é um ponto antes de uma instrução e p_{i+1} é um ponto depois de uma instrução; ou

Pontos e caminhos

- Para cada instrução do programa existe um ponto p_i antes de um ponto p_{i+1} depois da instrução.
- Um caminho de p_1 até p_n é uma sequência de pontos $p_1, p_2, \dots, p_n$ tal que para cada i entre 1 e $n - 1$:
 - p_i é um ponto antes de uma instrução e p_{i+1} é um ponto depois de uma instrução; ou
 - p_i é o ponto no fim de um bloco básico e p_{i+1} é um ponto no início de outro bloco básico.

Pontos e Caminhos



Análise de fluxo de dados

- Como Coletar informações?

Análise de fluxo de dados

- Como Coletar informações?
 - Resolvendo um sistema de equações de fluxo de dados.

Análise de fluxo de dados

- Como Coletar informações?
 - Resolvendo um sistema de equações de fluxo de dados.
 - Cada instrução s possui um conjunto de informações antes ($IN[s]$) e após ($OUT[s]$) a mesma.

Análise de fluxo de dados

- Como Coletar informações?
 - Resolvendo um sistema de equações de fluxo de dados.
 - Cada instrução s possui um conjunto de informações antes ($IN[s]$) e após ($OUT[s]$) a mesma.
 - Uma instrução s define restrições f_s entre os conjuntos $IN[s]$ e $OUT[s]$: Ex.: $OUT[s] = f_s(IN[s])$.

Análise de fluxo de dados

- Como Coletar informações?
 - Resolvendo um sistema de equações de fluxo de dados.
 - Cada instrução s possui um conjunto de informações antes ($IN[s]$) e após ($OUT[s]$) a mesma.
 - Uma instrução s define restrições f_s entre os conjuntos $IN[s]$ e $OUT[s]$: Ex.: $OUT[s] = f_s(IN[s])$.
 - Arestas do CFG da instrução s_i para s_j , definem restrições entre os conjuntos $OUT[s_i]$ e $IN[s_j]$.

Análise de fluxo de dados

- Exemplo: definições alcançantes no bloco básico

Análise de fluxo de dados

- Exemplo: definições alcançantes no bloco básico
 - $OUT[s] = gen[s] \cup (IN[s] - Kill[s]).$

$$a = b + c$$

$$b = a + d$$

$$a = a + b$$

Reaching Definitions

- Principal uso:

Reaching Definitions

- Principal uso:
 - Dada uma variável x e um certo ponto do programa

Reaching Definitions

- Principal uso:
 - Dada uma variável x e um certo ponto do programa
 - Inferimos que o valor de x é limitado a um determinado grupo de possibilidades.

Reaching Definitions

- Definição não ambígua de t :

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$
 - $d : t := M[a]$

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$
 - $d : t := M[a]$
- d alcança um uso na instrução u se:

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$
 - $d : t := M[a]$
- d alcança um uso na instrução u se:
 - existir um caminho no CFG da instrução que define d para u .

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$
 - $d : t := M[a]$
- d alcança um uso na instrução u se:
 - existir um caminho no CFG da instrução que define d para u .
 - esse caminho não contém outra definição não ambígua de t .

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$
 - $d : t := M[a]$
- d alcança um uso na instrução u se:
 - existir um caminho no CFG da instrução que define d para u .
 - esse caminho não contém outra definição não ambígua de t .
- Definição ambígua

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$
 - $d : t := M[a]$
- d alcança um uso na instrução u se:
 - existir um caminho no CFG da instrução que define d para u .
 - esse caminho não contém outra definição não ambígua de t .
- Definição ambígua
 - Uma sentença que pode ou não atribuir um valor a t .

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$
 - $d : t := M[a]$
- d alcança um uso na instrução u se:
 - existir um caminho no CFG da instrução que define d para u .
 - esse caminho não contém outra definição não ambígua de t .
- Definição ambígua
 - Uma sentença que pode ou não atribuir um valor a t .
 - CALL

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$
 - $d : t := M[a]$
- d alcança um uso na instrução u se:
 - existir um caminho no CFG da instrução que define d para u .
 - esse caminho não contém outra definição não ambígua de t .
- Definição ambígua
 - Uma sentença que pode ou não atribuir um valor a t .
 - CALL
 - Atribuição a ponteiros.

Reaching Definitions

- Definição não ambígua de t :
 - $d : t := a \text{ op } b$
 - $d : t := M[a]$
- d alcança um uso na instrução u se:
 - existir um caminho no CFG da instrução que define d para u .
 - esse caminho não contém outra definição não ambígua de t .
- Definição ambígua
 - Uma sentença que pode ou não atribuir um valor a t .
 - CALL
 - Atribuição a ponteiros.
 - Instrução com predicados.

Reaching Definitions

- Criamos identificadores para as definições

Reaching Definitions

- Criamos identificadores para as definições
 - $d_1 : t \leftarrow x \text{ op } y$

Reaching Definitions

- Criamos identificadores para as definições
 - $d_1 : t \leftarrow x \text{ op } y$
 - Gera d_1

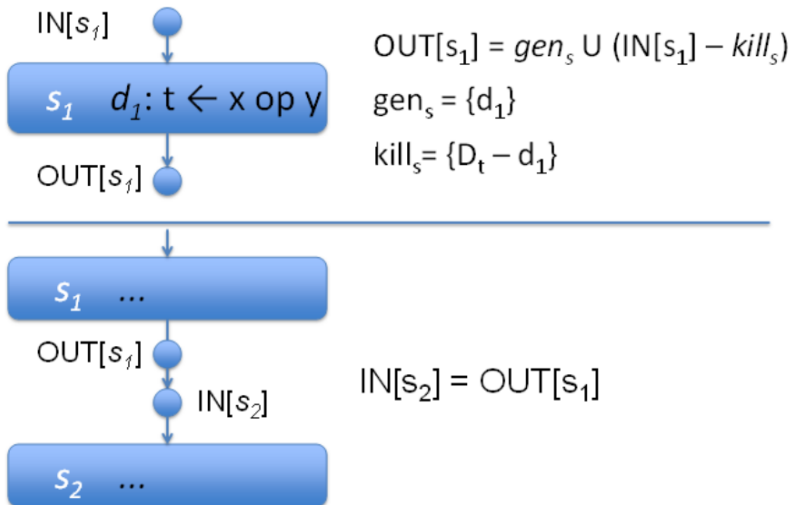
Reaching Definitions

- Criamos identificadores para as definições
 - $d_1 : t \leftarrow x \text{ op } y$
 - Gera d_1
 - Mata todas as outras definições de t , pois não alcançam o final dessa instrução

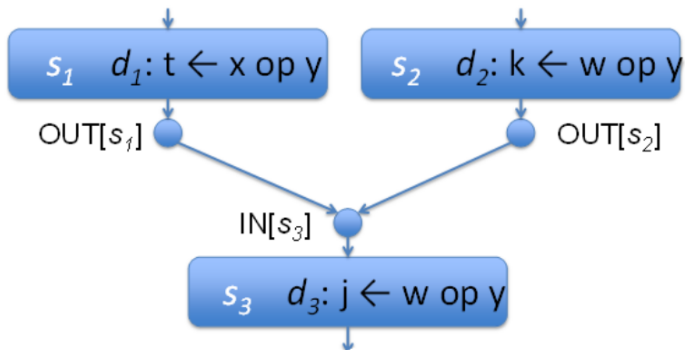
Reaching Definitions

- Criamos identificadores para as definições
 - $d_1 : t \leftarrow x \text{ op } y$
 - Gera d_1
 - Mata todas as outras definições de t , pois não alcançam o final dessa instrução
- $\text{defs}(t)$ ou D_t : conjunto de todas as definições de t no programa (ou função).

Reaching Definitions

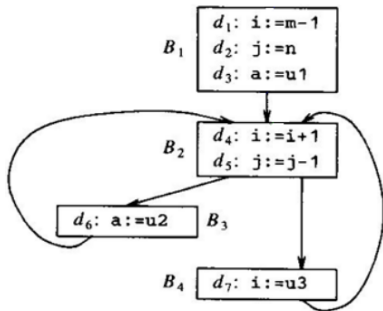


Reaching Definitions



Reaching Definitions

Exemplo



Definições	
i	
j	
a	

s	gen _s	kill _s	IN[s]	OUT[s]
1				
2				
3				
4				
5				
6				
7				

Reaching Definitions

- Agrupando equações por blocos básicos.

Reaching Definitions

- Agrupando equações por blocos básicos.
 - Em vez de $OUT[s] = f_s(IN[s])$.

Reaching Definitions

- Agrupando equações por blocos básicos.
 - Em vez de $OUT[s] = f_s (IN[s])$.
 - Fazemos: $OUT[B] = f_B (IN[B])$.

Reaching Definitions

- Agrupando equações por blocos básicos.
 - Em vez de $OUT[s] = f_s (IN[s])$.
 - Fazemos: $OUT[B] = f_B (IN[B])$.
- Reaching definitions em blocos básicos.

Reaching Definitions

- Agrupando equações por blocos básicos.
 - Em vez de $OUT[s] = f_s (IN[s])$.
 - Fazemos: $OUT[B] = f_B (IN[B])$.
- Reaching definitions em blocos básicos.
 - $OUT[B] = gen_B \cup (IN[B] - kill_B)$.

Reaching Definitions

- Agrupando equações por blocos básicos.
 - Em vez de $OUT[s] = f_s (IN[s])$.
 - Fazemos: $OUT[B] = f_B (IN[B])$.
- Reaching definitions em blocos básicos.
 - $OUT[B] = gen_B \cup (IN[B] - kill_B)$.
 - $IN[B] = \bigcup_p (p \text{ é predecessor de } B) OUT[P]$.

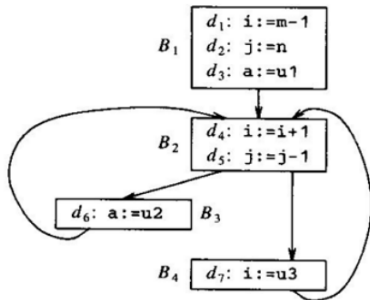
Reaching Definitions

- Agrupando equações por blocos básicos.
 - Em vez de $OUT[s] = f_s (IN[s])$.
 - Fazemos: $OUT[B] = f_B (IN[B])$.
- Reaching definitions em blocos básicos.
 - $OUT[B] = gen_B \cup (IN[B] - kill_B)$.
 - $IN[B] = \bigcup_p (p \text{ é predecessor de } B) OUT[P]$.
 - $Kill_B = kill_{s1} \cup kill_{s2} \dots \cup kill_{sn}$.

Reaching Definitions

- Agrupando equações por blocos básicos.
 - Em vez de $OUT[s] = f_s (IN[s])$.
 - Fazemos: $OUT[B] = f_B (IN[B])$.
- Reaching definitions em blocos básicos.
 - $OUT[B] = gen_B \cup (IN[B] - kill_B)$.
 - $IN[B] = \bigcup_p (p \text{ é predecessor de } B) OUT[P]$.
 - $Kill_B = kill_{s1} \cup kill_{s2} \dots \cup kill_{sn}$.
 - $gen_B = gen_{sn} \cup (gen_{sn-1} - kill_n) \cup \dots$

Reaching Definitions



Exemplo

Definições	
i	d_1, d_4, d_7
j	d_2, d_5
a	d_3, d_6

B	gen _B	kill _B	IN[B]	OUT[B]
1				
2				
3				
4				

Reaching Definitions = Algoritmo

$OUT[ENTRY] = \{\}$

for (each basic block B other than $ENTRY$)

$OUT[B] = \{\}$

while (changes to any OUT occur)

for (each basic block B other than $ENTRY$) {

$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P];$

$OUT[B] = gen_B \cup (IN[B] - kill_B);$

}

Reaching Definitions - Observações

- O algoritmo propaga as definições

Reaching Definitions - Observações

- O algoritmo propaga as definições
 - Até onde elas podem chegar sem serem mortas.

Reaching Definitions - Observações

- O algoritmo propaga as definições
 - Até onde elas podem chegar sem serem mortas.
 - Até que não haja mais modificações (todas as restrições são satisfeitas).

Reaching Definitions - Observações

- O algoritmo propaga as definições
 - Até onde elas podem chegar sem serem mortas.
 - Até que não haja mais modificações (todas as restrições são satisfeitas).
- O algoritmo sempre termina:

Reaching Definitions - Observações

- O algoritmo propaga as definições
 - Até onde elas podem chegar sem serem mortas.
 - Até que não haja mais modificações (todas as restrições são satisfeitas).
- O algoritmo sempre termina:
 - $OUT[B]$ nunca diminui de tamanho.

Reaching Definitions - Observações

- O algoritmo propaga as definições
 - Até onde elas podem chegar sem serem mortas.
 - Até que não haja mais modificações (todas as restrições são satisfeitas).
- O algoritmo sempre termina:
 - $OUT[B]$ nunca diminui de tamanho.
 - o número de definições é finito.

Reaching Definitions - Observações

- O algoritmo propaga as definições
 - Até onde elas podem chegar sem serem mortas.
 - Até que não haja mais modificações (todas as restrições são satisfeitas).
- O algoritmo sempre termina:
 - $OUT[B]$ nunca diminui de tamanho.
 - o número de definições é finito.
 - se OUT não muda, IN não muda no próximo passo.

Reaching Definitions - Observações

- O algoritmo propaga as definições
 - Até onde elas podem chegar sem serem mortas.
 - Até que não haja mais modificações (todas as restrições são satisfeitas).
- O algoritmo sempre termina:
 - $OUT[B]$ nunca diminui de tamanho.
 - o número de definições é finito.
 - se OUT não muda, IN não muda no próximo passo.
 - Limitante superior para número de iterações.

Reaching Definitions - Observações

- O algoritmo propaga as definições
 - Até onde elas podem chegar sem serem mortas.
 - Até que não haja mais modificações (todas as restrições são satisfeitas).
- O algoritmo sempre termina:
 - $OUT[B]$ nunca diminui de tamanho.
 - o número de definições é finito.
 - se OUT não muda, IN não muda no próximo passo.
 - Limitante superior para número de iterações.
 - Número de nós no CFG.

Reaching Definitions - Observações

- O algoritmo propaga as definições
 - Até onde elas podem chegar sem serem mortas.
 - Até que não haja mais modificações (todas as restrições são satisfeitas).
- O algoritmo sempre termina:
 - $OUT[B]$ nunca diminui de tamanho.
 - o número de definições é finito.
 - se OUT não muda, IN não muda no próximo passo.
 - Limitante superior para número de iterações.
 - Número de nós no CFG.
 - Pode ser melhorado de acordo com a ordem de avaliação dos nós.

Use-def Chains

- Armazenam a informação de *reaching*.

Use-def Chains

- Armazenam a informação de *reaching*.
- São listas para cada uso de uma variável contendo as definições que alcançam esse uso.

Use-def Chains

- Armazenam a informação de *reaching*.
- São listas para cada uso de uma variável contendo as definições que alcançam esse uso.
 - Considere variável a no bloco B.

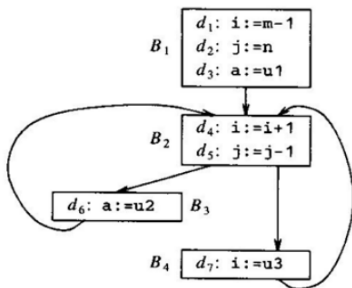
Use-def Chains

- Armazenam a informação de *reaching*.
- São listas para cada uso de uma variável contendo as definições que alcançam esse uso.
 - Considere variável a no bloco B .
 - Se B não contém definições de a , *ud-chain* é o conjunto de definições de a em $\text{in}[B]$.

Use-def Chains

- Armazenam a informação de *reaching*.
- São listas para cada uso de uma variável contendo as definições que alcançam esse uso.
 - Considere variável a no bloco B.
 - Se B não contém definições de a , *ud-chain* é o conjunto de definições de a em $\text{in}[B]$.
 - Se B contém definições de a , então a *ud-chain* é a última dessas definições, antes do uso.

Exemplo



B	IN[B]	OUT[B]
1		d1, d2, d3
2	d1, d2, d3, d6	d3, d4, d5, d6
3	d3, d4, d5, d6	d4, d5, d6
4	d3, d4, d5, d6	d3, d5, d6, d7

Exemplo

ud-chain	

Seção 1

Análise de Fluxo de dados

Available Expressions

- Expressão disponível:

Available Expressions

- Expressão disponível:
 - $x+y$ está disponível em p se:

Available Expressions

- Expressão disponível:
 - $x+y$ está disponível em p se:
 - todo caminho do nó inicial até p calcula $x+y$.

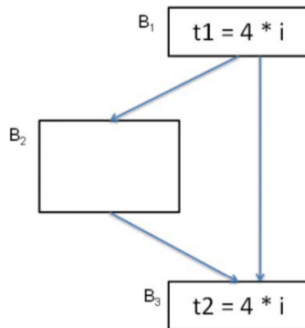
Available Expressions

- Expressão disponível:
 - $x+y$ está disponível em p se:
 - todo caminho do nó inicial até p calcula $x+y$.
 - após a última computação de $x+y$, nem x nem y sofrem atribuições.

Available Expressions

Instruções	Expressões disponíveis
$a = b + c$	
$b = a - d$	
$c = b + c$	
$d = a - d$	

Available Expressions



Available Expressions

- Kill:

Available Expressions

- Kill:
 - Um bloco B mata, ou pode matar, $x+y$ se ele atribui a x e/ou y , e não recomputa $x+y$.

Available Expressions

- Kill:
 - Um bloco B mata, ou pode matar, $x+y$ se ele atribui a x e/ou y , e não recomputa $x+y$.
- Gen:

Available Expressions

- Kill:
 - Um bloco B mata, ou pode matar, $x+y$ se ele atribui a x e/ou y , e não recomputa $x+y$.
- Gen:
 - Um bloco B gera $x+y$ se ele certamente computa $x+y$, e não redefine x ou y .

Equações da DFA

- Computamos gen e kill para cada B como visto anteriormente.
- Temos:

Equações da DFA

- Computamos gen e kill para cada B como visto anteriormente.
- Temos:

$\text{out}[\text{ENTRY}] = \emptyset$

Para todo bloco B diferente de ENTRY

$\text{out}[B] =$

$\text{in}[B] =$

Equações da DFA

- Computamos gen e kill para cada B como visto anteriormente.
- Temos:

$$\text{out}[\text{ENTRY}] = \emptyset$$

Para todo bloco B diferente de ENTRY

$$\text{out}[B] =$$

$$\text{in}[B] = \bigcap_{p \in \text{pod}} \text{out}[p]$$

Equações da DFA

- Computamos gen e kill para cada B como visto anteriormente.
- Temos:

$$\text{out}[\text{ENTRY}] = \emptyset$$

Para todo bloco B diferente de ENTRY

$$\text{out}[B] = \text{gen}[B] \cup (\text{in}[B] - \text{kill}[B])$$

$$\text{in}[B] = \bigcap_{p \in \text{pod}} \text{out}[p]$$

Available Expressions

$$out[ENTRY] = \emptyset$$

$$out[B] = gen[B] \cup (in[B] - kill[B])$$

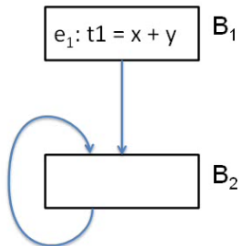
$$in[B] = \bigcap_{P \in \text{predecessores de } B} out[P]$$

$$in[B_1] =$$

$$out[B_1] =$$

$$in[B_2] =$$

$$out[B_2] =$$



Available Expressions

$$out[ENTRY] = \emptyset$$

$$out[B] = gen[B] \cup (in[B] - kill[B])$$

$$in[B] = \bigcap_{P \in \text{predecessores de } B} out[P]$$

$$in[B_1] = \{\}$$

$$out[B_1] = \{e_1\}$$

$$in[B_2] = out[B_1] \cap out[B_2]$$

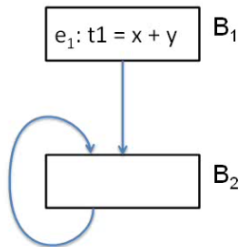
$$= \{e_1\} \cap \{\}$$

$$= \{\}$$

$$out[B_2] = gen[B_2] \cup (in[B_2] - kill[B_2])$$

$$= \{\} \cup (\{\} - \{\})$$

$$= \{\}$$



Available Expressions

$$out[ENTRY] = \emptyset$$

$$out[B] = gen[B] \cup (in[B] - kill[B])$$

$$in[B] = \bigcap_{P \in \text{predecessores de } B} out[P]$$

// Iniciar out[B] com todas as expressões

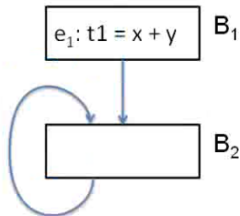
$$out[B_1] = \{e_1\}$$

$$out[B_2] = \{e_1\}$$

$$in[B_1] =$$

$$in[B_2] =$$

$$out[B_2] =$$



Available Expressions - Algoritmo

$OUT[ENTRY] = gen[ENTRY]; IN[ENTRY] = \{\}$

for (each basic block B other than $ENTRY$)

$OUT[B] = U - e_kill[B]$ // $U =$ Todas as expressões

while (changes to any OUT occur)

for (each basic block B other than $ENTRY$) {

$$in[B] = \bigcap_{P \in \text{predecessores de } B} out[P]$$

$$out[B] = gen[B] \cup (in[B] - kill[B])$$

}

Diferenças para *Reaching Defs*

- O operador de confluência é intersecção.

Diferenças para *Reaching Defs*

- O operador de confluência é intersecção.
 - Tem que vir por todos os caminhos.

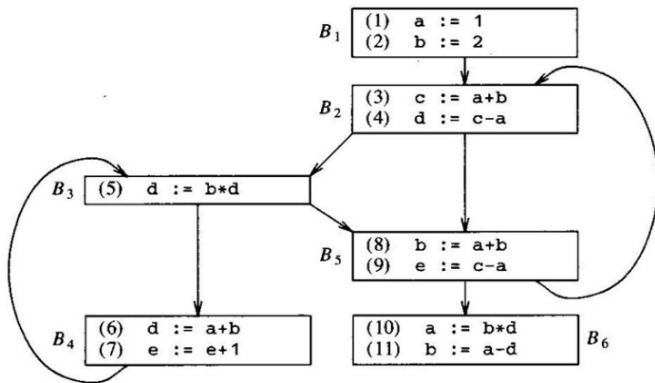
Diferenças para *Reaching Defs*

- O operador de confluência é intersecção.
 - Tem que vir por todos os caminhos.
- Estimativa inicial é grande

Diferenças para *Reaching Defs*

- O operador de confluência é intersecção.
 - Tem que vir por todos os caminhos.
- Estimativa inicial é grande
 - Intersecção vai diminuindo os conjuntos até chegar ao maior ponto fixo.

Available Expressions - Exemplo



Análise de Fluxo de Dados



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismailly
ismailybf@ufc.br

Semestre 20025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)